

was in conditional wait, it can now wake up automatically (the mutex will be locked) and proceed.

You may wonder why the *done* variable needs to be checked again? This is a recommended practice, since there can be spurious wake-ups too. (You may download the file http://www.linuxforu.com/article_source_code/oct12/system_programming.zip, to experiment with the condition variables.)

Reentrancy and thread safety

When writing code, say a function, for a multi-threaded application, the function must be thread-safe. This means that the function should be logically correct, so that you get proper output when it is executed simultaneously by multiple threads. You need to be careful that your code does not lead to a sudden runtime error, or incorrect results due to threading. These types of errors may not get detected while you are compiling your code and are usually due to the incorrect design of the problem statement. Observe the code below:

```
char *strconvert(char *string) {
    static char buffer[MAX_STRING];
    int index;
    for (index = 0; string[index]; index++)
        buffer[index] = tolower(string[index]);
    buffer[index] = '\0';

    return buffer;
}
```

The function does some conversion (here, converting to lowercase) on an input string, and

the resultant string is passed back to the caller. This is fine when you are using this in a single-threaded program. But can you use this in a multi-threaded program, where it can be called simultaneously by multiple threads? No, you cannot, which is quite interesting. The function uses a static array—so it can happen that while one thread is running this function, it gets pre-empted, and another thread gets scheduled. The second thread also calls the function. Now the array buffer is a static one, and hence its state gets changed. So when the earlier thread gets scheduled again, it will access the buffer, which is now changed—and hence yield a wrong output.

If you experiment with this code, you can see that perhaps you are getting correct results (for small strings) since one thread that was running this function did not get pre-empted. In spite of this, you cannot assume that this will work correctly for all cases.

Such functions are called non-reentrant functions, and are unsuitable for multi-threaded programs. This is quite a big problem. To solve it, you can change the interface of the function as follows:

```
char *strconvert_r(char *input, char *output) {
    int index;
```

```
    for (index = 0; input[index]; index++)
        output[index] = toupper(input[index]);
    output[index] = '\0';
    return output;
}
```

You will notice that the caller allocates memory here in the variable *output*. Even if multiple threads are calling this function, all the variables that are used are local—so the context can be saved for all calls, and thus no non-deterministic or incorrect result will occur. Also, in production code, you need more checks and appropriate error handling, like checking the size of the output string, if it is big enough to hold the (transformed) input string. Doing all these for a very large program is quite a large task, and we need to have a proper design to make robust multi-threaded applications. You will be surprised to know that most of the C library functions are unsuitable for use in multi-threaded programs. In the standard library, you should check for functions that are reentrant—such as *strtok*, which is commonly used for string tokenising and is a non-reentrant function. But if you use the *strtok_r* function, it is reentrant. The interfaces of *strtok* and *strtok_r* differ. In the manual page, I see the warning, “*The strtok() function uses a static buffer while parsing, so it's not thread safe. Use strtok_r() if this matters to you.*”

Debugging multi-threaded programs using GDB

Take note of some of the debugger commands that can help you debug multi-threaded programs. I have used GDB release 7.2 on Fedora 15. From the output snippets below, you can see that one breakpoint has been set in the function *thread_entry*, and then execution has stopped. The *info threads* command shows the current state of the threads (here, three threads), with an identifier associated with each thread. The * indicates the current thread.

```
Fedora$ gcc lfy_thread_2.c -o mythread -Wall -g -lpthread
Fedora$ gdb ./mythread
```

```
(gdb) set target-async on
(gdb) set non-stop on
```

The above two GDB settings need to be turned on for debugging, manipulating or examining individual threads under GDB. In *non-stop* mode, when a thread stops, only that thread is stopped. The debugger does not stop other threads as well. The implication of *target-async* on is that instead of waiting for a command to be completed, the debugger gives you the prompt back, and runs some command in the background. If you notice the snippets in the last section, I have used ‘c&’, which means that the current thread can continue executing in the background while GDB can accept subsequent commands.